LEVELUP CLOUD TRAININGS

PySpark RDD Masterclass

Module 6 — Industry Interview Assignment

SaveAsTextFile | coalesce | repartition | foreach | pipe | join | union | mapPartitions

Questions	Domains	Difficulty	Style
40	8	Easy → Hard	Interview

How to Use This Assignment

Each question is tagged with: Question Type (Conceptual / Code Trace / Scenario / Design) • Difficulty Level (Easy / Medium / Hard) • Company Context (real Indian/global companies). Answer each question as you would in a real interview — clearly explain your thought process, give examples, and use code where asked. Model answers are provided for self-evaluation.

Section 1: saveAsTextFile

4 Questions | Conceptual · Scenario · Design · Code Trace

Q1 Conceptual ★ Easy — TCS Data Engineering L1

Explain what saveAsTextFile() does in PySpark. What does the output directory look like and why does it produce multiple part files?

Hint: *Think about how partitions map to output files.*

✓Model Answer (Key Points to Cover):

saveAsTextFile() is an ACTION that serializes each RDD element as a string and writes it to a distributed file system (HDFS/S3/local). Each partition writes independently, producing one part-XXXXX file per partition. For example, an RDD with 4 partitions creates part-00000 through part-00003. This parallel write is why Spark is fast — all partitions write simultaneously without bottlenecking through a single writer.

Q2 Scenario ★ Medium — Infosys — Fintech Client Project

You are building a daily EOD (End of Day) report pipeline for an HDFC Bank settlement system. The pipeline processes 50 million transaction records and must write the final output to HDFS as exactly 5 files (one per zone: North, South, East, West, Central). How do you achieve this using saveAsTextFile?

Hint: *Think: control output file count, partition by zone, and then save.*

✓Model Answer (Key Points to Cover):

Step 1: After processing, group by zone using a key. Step 2: Use repartition(5) or coalesce(5) to reduce to exactly 5 partitions. Step 3: Call saveAsTextFile('/hdfs/eod_report'). Prefer coalesce(5) here since we are reducing partitions and want to avoid a full shuffle. Important: saveAsTextFile fails if the output directory already exists — delete or use a timestamped path in production. Also add GzipCodec compression for large files: saveAsTextFile(path, GzipCodec).

Q3 Code Trace ★ Medium — Wipro — Data Lake Project

What is wrong with the following production code? How will you fix it?

```
from pyspark import SparkContext
sc = SparkContext('yarn', 'EODJob')
transactions = sc.textFile('/data/input/transactions.csv')
processed = transactions.map(lambda x: x.split(',')[0] + ',' + x.split(',')[3])
processed.saveAsTextFile('/data/output/eod_report')  # Runs daily!
```

Hint: *Look at what happens if the job runs twice. What does Spark do when output path already exists?*

✓Model Answer (Key Points to Cover):

Bug: If this job runs daily, on day 2 it fails with 'Output directory already exists' error because saveAsTextFile() cannot overwrite. Fix: Either (a) delete the existing directory before saving:

sc._jvm.org.apache.hadoop.fs.FileSystem.get(...).delete(path, True), or (b) use a dynamic timestamp in the output path: f'/data/output/{datetime.now().strftime("%Y%m%d")}', or (c) switch to DataFrame's .write.mode('overwrite').text(path) which supports overwrite mode.

Q4 | Design | ★ Hard | Capgemini — Insurance Data Platform

Design a PySpark pipeline for LIC (Life Insurance Corp) that processes 200 million policy records output must be partitioned by policy_type (TERM, ULIP, ENDOWMENT, WHOLE_LIFE) and written files to S3. What approach would you use and why?

Hint: *Consider: partition control, file format, compression, scalability. Think DataFrame write vs RDD saveAsTextFile.*

✅Model Answer (Key Points to Cover):

Production answer: Prefer DataFrame's .write API over RDD saveAsTextFile for this scale. Use:
df.repartition(col('policy_type')).write.partitionBy('policy_type').mode('overwrite').option('compression','snappy').parquet('s
If forced to use RDD + saveAsTextFile: filter by type, coalesce(20) per type, save to type-specific paths with GzipCodec.
200M records, Parquet + Snappy is 70-80% smaller and 5x faster to query than plain text. Always justify format choice in
shows senior-level thinking.

Section 2: coalesce & repartition

6 Questions | Conceptual · Scenario · Code Trace · Design

Q5 Conceptual ★ Easy — TCS / Accenture L1 Interview

What is the fundamental difference between coalesce() and repartition()? When would you choose one over the other?

Hint: *Think about shuffle, data movement, and direction (increasing vs decreasing).*

✅Model Answer (Key Points to Cover):

coalesce(n): Reduces partitions WITHOUT a full shuffle. It merges nearby partitions on the same node. Fast but can create uneven partition sizes (skew). Use when DECREASING partitions — especially before saveAsTextFile to control output file count. repartition(n): Triggers a FULL SHUFFLE — redistributes all data evenly. Works for both increase and decrease. Use when INCREASING partitions (before joins/aggregations) or when you need perfectly balanced partitions. Golden rule: coalesce to shrink, repartition to grow or rebalance.

Q6 Scenario ★ Medium — Flipkart — Big Sale Event Pipeline

During Flipkart's Big Billion Day, your PySpark job starts with 500 partitions (auto-created from 500GB input). After heavy filtering, only 2GB of data remains. The job then does a groupByKey. Your colleague says 'just use groupByKey — the data is smaller now'. What is the performance problem and how do you fix it?

Hint: *What happens when 500 partitions have mostly empty data? Think shuffle overhead.*

✅Model Answer (Key Points to Cover):

Problem: Even though only 2GB remains, Spark still schedules 500 tasks for groupByKey — 490+ tasks are near-empty, creating massive overhead from task scheduling, shuffle bookkeeping, and small file writes. Fix: After the filter, call coalesce(20) to reduce to a reasonable partition count before groupByKey. Rule of thumb: target 128-256MB per partition. 2GB / 128MB = 16 partitions, so coalesce(16) is appropriate. This reduces shuffle overhead drastically. This is a classic 'partition tuning' question at Flipkart/Amazon interviews.

Q7 Code Trace ★ Medium — Infosys — Spark Performance Round

Analyze this code. Will it work correctly? What is the output partition count and is there any performance concern?

```
# Spark job for joining Swiggy restaurant data
restaurant_rdd = sc.parallelize(restaurant_data, 10)  # 10 partitions
order_rdd = sc.parallelize(order_data, 500)           # 500 partitions

# Developer wants to increase parallelism for better join performance
restaurant_rdd_scaled = restaurant_rdd.coalesce(200)
print(restaurant_rdd_scaled.getNumPartitions())  # What prints?
joined = restaurant_rdd_scaled.join(order_rdd)
```

Hint: *coalesce() cannot increase partitions. What happens when you try?*

✅Model Answer (Key Points to Cover):

Behavior: coalesce(200) on a 10-partition RDD will NOT increase to 200. coalesce() can only DECREASE or MAINTAIN partition count. With 10 partitions, coalesce(200) silently returns 10 partitions. Performance concern: The developer intended to increase parallelism before the join but used the wrong function. Fix: Use repartition(200) to actually increase partitions. Lesson: Always verify partition count with .getNumPartitions() in tests. This subtle bug causes silent underperformance — a classic interview gotcha.

Q8 | **Scenario** | **★ Hard** | Paytm — Payment Processing Pipeline

Paytm processes UPI payments across 28 states. The raw RDD has 1000 partitions. You need to: (1) filter only failed transactions, (2) group by state to get failure counts, (3) write one file per state to HDFS. Walk through the exact PySpark steps with partition strategy at each stage.

Hint: *Think: filter reduces data volume → rebalance → aggregate → final partition count for write.*

✅Model Answer (Key Points to Cover):

Step 1: After filter (say 10% failures remain), call coalesce(100) — reduce from 1000 to 100 since data shrank 90%. Step 2: Call repartition(28) before groupByKey — ensures each state gets its own partition and data is evenly distributed (full shuffle needed here for even distribution). Step 3: groupByKey() or reduceByKey() to aggregate per state. Step 4: coalesce(1) per state group OR call saveAsTextFile with 28 partitions to get 28 output files. Interview answer tip: Always explain the WHY behind each partition decision — that's what separates a good engineer from a great one.

Q9 | **Conceptual** | **★ Medium** | Wipro / HCL Technical Interview

What is partition skew? How does repartition() help solve it? Give a real-world example from an e-commerce company.

Hint: *Think about uneven data distribution and its impact on job completion time.*

✅Model Answer (Key Points to Cover):

Partition skew: When some partitions have significantly more data than others, causing a few slow tasks (stragglers) to delay the entire job. Real example: Meesho processes orders by city. Mumbai alone generates 40% of all orders. After groupByKey on city, Mumbai's partition has 40% of the data while all other partitions have 60% split across 27 cities — Mumbai's task takes 4x longer, blocking the job. Fix: Use repartition(by_hash_of_random_salt) to artificially break the skew, or use salting technique: add a random prefix to the key before grouping, group, then aggregate again removing the salt. repartition alone (without salting) won't fix key-level skew.

Q10 | **Design** | **★ Hard** | Amazon — Data Engineering Manager Round

You are designing a PySpark pipeline that ingests 1TB of raw clickstream data from Amazon's website every hour. The pipeline applies 15 transformations, then writes to 3 downstream systems: (a) HDFS for data lake (want 1000 files), (b)

Elasticsearch (want small batches), (c) A summary report to S3 (want 1 file per category). Design the partition strategy for each output.

 Hint: *Each output has different needs — think partition count independently for each branch using cache().*

✅Model Answer (Key Points to Cover):

Architecture: After the 15 transformations, call rdd.cache() to avoid recomputation for 3 outputs. Then branch: (a) HDFS: rdd.repartition(1000).saveAsTextFile(hdfs_path) — full repartition for even distribution across 1000 files. (b) Elasticsearch: rdd.coalesce(200).foreachPartition(lambda p: es_bulk_insert(p)) — small partitions, use foreachPartition for connection reuse. (c) S3 summary: rdd.map(categorize).reduceByKey(agg).coalesce(1).saveAsTextFile(s3_path) — one file after aggregation. Key: Always cache() before multiple actions on same RDD — without cache, all 15 transformations replay 3 times!

⚡ Section 3: foreach & pipe

5 Questions | Conceptual · Scenario · Design · Debug

Q11 Conceptual ★ Easy Cognizant L1 Interview

What is the difference between map() and foreach() in PySpark? When would you use foreach() instead of map()?

 Hint: *Think about return value, transformation vs action, and side effects.*

✅Model Answer (Key Points to Cover):

map() is a TRANSFORMATION — it applies a function to each element and returns a NEW RDD. It's lazy (executes only when an action is called). foreach() is an ACTION — it applies a function to each element but returns NOTHING (Unit). It is used purely for SIDE EFFECTS: writing to databases, logging, calling external APIs. Use foreach when you don't need the result back — you're pushing data OUT of Spark into another system. Example: inserting records into MySQL after processing, publishing to Kafka, writing to Redis cache.

Q12 Scenario ★ Medium Zomato — Real-Time Order Processing

Zomato's data team processes confirmed orders and needs to insert each order record into a MySQL database. A junior developer wrote: rdd.foreach(lambda order: insert_to_mysql(order)). This works but the performance is terrible in production. What is the problem and what is the correct solution?

 Hint: *Think about connection overhead. A new DB connection per record vs per partition.*

✅Model Answer (Key Points to Cover):

Problem: The code opens and closes a MySQL connection for EVERY single record. If there are 10 million orders, that's 10 million connection open/close operations — catastrophically slow and likely crashes the MySQL connection pool. Solution: Use foreachPartition() instead. foreachPartition gives you an iterator over an entire partition, so you open ONE connection per partition, insert all records in batch, then close. Code: rdd.foreachPartition(lambda partition: batch_insert_to_mysql(partition)). With 100 partitions of 100K records each, you use 100 connections total instead of 10 million. This is a CRITICAL interview question at product companies.

Q13 Code Trace ★ Medium Tech Mahindra — Spark Interview

A developer wants to use foreach() to count records processed. Will this code work correctly? Explain why or why not.

```
count_processed = 0  # Regular Python variable on driver

def process_and_count(record):
    # Do some DB write here...
    count_processed += 1  # Trying to count on driver

orders_rdd.foreach(process_and_count)
print(f'Processed: {count_processed}')  # Always prints 0 !
```

Hint: *Recall the rules about reading variables modified inside tasks.*

✅Model Answer (Key Points to Cover):

This code is WRONG. count_processed is a regular Python variable on the driver. When PySpark ships the lambda to executors, each executor gets its own LOCAL copy of count_processed and increments it. These local copies are NEVER sent back to the driver. After collect/foreach, count_processed on the driver remains 0. Fix: Use a PySpark Accumulator instead: count_acc = sc.accumulator(0), then inside foreach: count_acc.add(1). After the action, read count_acc.value on the driver. This is a classic interview trap question.

Q14 | **Conceptual** | **★ Medium** | Accenture — Advanced Spark Round

Explain the pipe() function in PySpark with a real use case. When would you prefer pipe() over writing a native PySpark transformation?

Hint: *Think about legacy systems, external scripts, shell commands.*

✅Model Answer (Key Points to Cover):

pipe() sends each partition's data as stdin to a shell command and captures stdout as a new RDD. Each partition runs the command as an independent process. Real use case: A telecom company (Airtel) has a legacy Perl script that applies complex number formatting/validation rules for mobile numbers — rewriting this in PySpark would take months. Using pipe(), they send each partition to the Perl script and get validated numbers back. When to prefer pipe(): (1) Existing battle-tested scripts you don't want to rewrite, (2) Tools with no Python API (specialized C++ analytics tools), (3) Rapid integration prototype. Caution: pipe() has overhead (process spawn per partition) and is hard to debug — use only when genuinely needed.

Q15 | **Design** | **★ Hard** | Infosys BPM — Data Quality Pipeline

Design a real-time data quality pipeline for ICICI Bank's transaction system using foreach/foreachPartition. The pipeline must: validate 5 million transactions/hour, send invalid records to a Kafka dead-letter topic, update a Redis counter with error counts, and log summaries to Elasticsearch. How do you structure this with Spark Streaming + foreach?

Hint: *Think about connection pooling, partition-level efficiency, and resource management.*

✅Model Answer (Key Points to Cover):

Architecture: Use Spark Structured Streaming with foreachBatch for micro-batches. Each micro-batch: (1) Validate in parallel using map() with validation rules. (2) Split valid/invalid using filter() on a flag column. (3) Invalid records: foreachPartition(lambda p: produce_to_kafka(p)) — one Kafka producer per partition, not per record. (4) Error count: Use accumulators or foreachBatch-level aggregation, then foreachPartition to push to Redis (one Redis connection per partition). (5) Summary: collect aggregated metrics to driver, then single Elasticsearch insert. Connection pooling pattern: Use a connection factory inside foreachPartition that checks for existing connection before creating new one — prevents connection storms during recovery.

Section 4: join & union

7 Questions | Conceptual · Scenario · Design · Performance

Q16 Conceptual ★ Easy Wipro / TCS L1

List all join types available in PySpark RDD. What is the default join type and what does it return?

Hint: *Think: inner, left, right, full, cartesian. What happens to non-matching keys?*

Model Answer (Key Points to Cover):

PySpark RDD supports: (1) join() — INNER JOIN (default): only keys in BOTH RDDs. Returns (key, (v1, v2)). (2) leftOuterJoin() — all keys from LEFT, None for missing right values. (3) rightOuterJoin() — all keys from RIGHT, None for missing left values. (4) fullOuterJoin() — all keys from both, None where missing. (5) subtractByKey() — keys in LEFT not in RIGHT. (6) cartesian() — every combination of left × right (VERY expensive, rarely used). Default is inner join. In DataFrame API, additional types like semi-join and anti-join are also available.

Q17 Scenario ★ Medium Flipkart — Seller Analytics Team

Flipkart has a seller_rdd (5 million sellers, 2GB) and a product_rdd (500 million products, 200GB). You need to enrich each product with seller information. A standard join() causes a massive shuffle and OOM errors. What is the production solution?

Hint: *One RDD is small (2GB fits in memory). Think: broadcast join pattern.*

Model Answer (Key Points to Cover):

Solution: Broadcast Join. Since seller_rdd is only 2GB, collect it to the driver and broadcast it: seller_bc = sc.broadcast(dict(seller_rdd.collect())). Then use map() on product_rdd to look up seller info: enriched = product_rdd.map(lambda p: (p[0], p[1], seller_bc.value.get(p[0]))). No shuffle! Every executor has a local copy of the seller data. Performance: Standard join on 200GB triggers a 200GB shuffle across the network. Broadcast join has ZERO shuffle. This is one of the most asked performance optimization questions at Flipkart/Amazon/Meesho.

Q18 Code Trace ★ Medium HCL — Spark Technical Round

Predict the output of this leftOuterJoin. How many records will the result have? What are the exact values?

```
customers = sc.parallelize([(1,'Rahul'), (2,'Priya'), (3,'Arjun')])
orders    = sc.parallelize([(1, 1200.0), (1, 800.0), (3, 3500.0), (4, 200.0)])

result = customers.leftOuterJoin(orders)
print(sorted(result.collect()))
# How many records? What are the exact tuples?
```

Hint: *leftOuterJoin keeps ALL records from the left RDD. Missing right values become (key, (left_val, None)).*

✅Model Answer (Key Points to Cover):

Result has 4 records (one per customer, since we keep ALL customers): (1, ('Rahul', 1200.0)), (1, ('Rahul', 800.0)) — customer 1 has 2 orders, (2, ('Priya', None)) — customer 2 has no orders, left join preserves her with None, (3, ('Arjun', 3500.0)) — customer 3 has one order. Note: customer 4 in orders (no customer record) is dropped since this is a LEFT outer join. Total: 4 records.

Q19 | Scenario | ★ Hard | Amazon India — Senior Data Engineer

You are joining Amazon's 10TB customer transaction history with a 500GB product catalog. Both datasets are large (no broadcast possible). The join keeps timing out due to data skew — 60% of transactions are for 10 top-selling products. Design a salting strategy to fix this skew.

Hint: *Salting adds a random suffix to skewed keys to distribute them across partitions.*

✅Model Answer (Key Points to Cover):

Salting technique: (1) Identify skewed keys (top 10 products). (2) For product_rdd (small side), replicate each skewed key N times with suffixes: (prod_id_0, data), (prod_id_1, data)...(prod_id_N, data) where N = salt_factor (e.g., 50). (3) For transaction_rdd (large side), add random salt to skewed keys: (prod_id_random(0-49), data). (4) Now join on salted keys — traffic is spread across 50 partitions per product instead of 1. (5) After join, strip the salt from the key. Non-skewed keys: join normally or use a separate join and union the results. Code: key = f'{prod_id}_{random.randint(0, salt_factor-1)}'. This is a Senior DE level question — knowing salting separates L5 from L6 at Amazon.

Q20 | Conceptual | ★ Easy | Cognizant / Mphasis Interview

What is the difference between union() and join() in PySpark? Give a practical example of when you would use union().

Hint: *union stacks rows vertically; join combines columns horizontally.*

✅Model Answer (Key Points to Cover):

join(): Combines two RDDs HORIZONTALLY based on matching keys — like SQL JOIN. Result has columns from BOTH RDDs for matching keys. union(): Combines two RDDs VERTICALLY by appending all rows — like SQL UNION ALL. Both RDDs must have the same schema/structure. No key matching happens. Practical example: HDFC Bank runs separate daily processing for Debit and Credit card transactions. At EOD, debit_rdd.union(credit_rdd) creates one combined RDD for the daily settlement report. Note: union() in RDD is equivalent to UNION ALL in SQL — duplicates are preserved. To get UNION DISTINCT behavior, call .distinct() after union().

Q21 | Scenario | ★ Medium | Myntra — ETL Pipeline Interview

Myntra loads product data from 4 regional warehouses (North, South, East, West) stored as separate RDDs. They need a single consolidated RDD for nationwide inventory analysis. What approach do you take? What if duplicates exist across regions?

Hint: *Think: multiple unions, then deduplication strategy.*

✅Model Answer (Key Points to Cover):

Step 1: Combine all 4 RDDs using union: all_products = north.union(south).union(east).union(west). Step 2: Handle duplicates. If a product appears in 2 warehouses, we need to decide: (a) Keep all records (stock count per warehouse): use union as-is. (b) Get unique products only: call all_products.distinct() — removes exact duplicates. (c) Sum stock across warehouses: convert to (product_id, stock) pairs and use reduceByKey(lambda a,b: a+b) to aggregate. Interview tip: Always clarify the requirement — 'consolidate' is ambiguous. Do they want all records, unique records, or aggregated records? Asking this clarifying question demonstrates senior-level thinking.

Q22 | **Design** | **★ Hard** | Razorpay — Data Platform Architecture

Design a reconciliation pipeline for Razorpay. Every day, 50 million payment records come from 3 sources: (1) internal_payments_rdd from their database, (2) bank_settlements_rdd from partner banks, (3) upi_transactions_rdd from NPCI. You must find: (a) payments in internal but NOT in bank, (b) payments in bank but NOT in internal, (c) payments in all 3 systems (fully reconciled). Design the join/union strategy.

Hint: *Use subtractByKey for mismatches, join for intersection. Think about key selection.*

✅Model Answer (Key Points to Cover):

Key on transaction_id. (a) Internal NOT in bank: internal_payments_rdd.subtractByKey(bank_settlements_rdd) — gives discrepancies (missing settlements). (b) Bank NOT in internal: bank_settlements_rdd.subtractByKey(internal_payments_rdd) — gives ghost transactions. (c) All 3 reconciled: internal_payments_rdd.join(bank_settlements_rdd).join(upi_transactions_rdd) — chained inner join. For production: cache internal_payments_rdd (used 3 times) before operations. Write results to separate HDFS paths per category. Alert finance team via foreach+Kafka for category (a) and (b) mismatches. This architecture handles the core of any payment reconciliation system at scale.

Section 5: mapPartitions

6 Questions | Conceptual · Performance · Design · Scenario

Q23 Conceptual ★ Easy TCS / Capgemini L1

What is mapPartitions() in PySpark? How is it different from map()? When should you prefer mapPartitions over map?

Hint: *Think about per-record vs per-partition processing. Connection overhead.*

Model Answer (Key Points to Cover):

map() processes ONE element at a time — function is called N times for N elements. mapPartitions() processes one PARTITION at a time — function receives an ITERATOR over all elements in the partition. Key difference: With map(), if you open a DB connection inside the function, it opens for every single record. With mapPartitions(), you open the connection ONCE per partition, process all records, close. Use mapPartitions when: (1) The setup cost is high (DB connections, ML model loading, file handles), (2) You need to batch process records together, (3) You want to reduce per-record overhead. Downside: If a partition is very large, all its data must fit in memory.

Q24 Code Trace ★ Medium Infosys — Technical Screening

What does this mapPartitions code do? What is the output? Why is this better than using map() for this task?

```
def score_partition(records):
    import pickle
    model = pickle.load(open('credit_model.pkl','rb'))  # Load ONCE
    for cust_id, features in records:
        score = model.predict([features])[0]
        yield (cust_id, score)  # yield makes it an iterator

customer_rdd = sc.parallelize([
    ('C001', [750, 5, 12, 0.3]),
    ('C002', [620, 2, 36, 0.7]),
    ('C003', [810, 8, 6, 0.1])
])
scored = customer_rdd.mapPartitions(score_partition)
print(scored.collect())
```

Hint: *mapPartitions gives you an iterator. You must return an iterator from your function.*

Model Answer (Key Points to Cover):

The code uses mapPartitions to score each partition's records using a machine learning model. The model is loaded ONCE per partition (not per record). For a partition with 100K records: map() approach would load the model 100K times. mapPartitions() loads it ONCE and scores all 100K records. Output: an RDD of (customer_id, credit_score) tuples. Why better: Loading a typical ML model (even a small scikit-learn model) takes 50-200ms. With map() and 10M records: 10M × 100ms = 10M+ seconds. With mapPartitions and 100 partitions: 100 × 100ms = 10 seconds. Massive difference!

Q25 Scenario ★ Medium PhonePe — ML Scoring Pipeline

PhonePe needs to run fraud detection on 50 million UPI transactions per hour. The fraud model is a 200MB TensorFlow model. The pipeline uses map() currently and takes 6 hours to process. Your job is to reduce this to under 30 minutes. What is your approach?

 Hint: *200MB model load time is the bottleneck. Think: load once, score many.*

✅Model Answer (Key Points to Cover):

Root cause: 200MB model loaded 50 million times = ~50M × 500ms = catastrophic. Solution: Use mapPartitions with model caching. Optimal approach: (1) Increase parallelism: repartition(500) to create 500 partitions. (2) mapPartitions with model loaded once per partition: each partition has 100K records, model loaded 500 times total. (3) Use broadcast for model weights OR let each executor load from shared storage. (4) Consider foreachPartition + external scoring service if model is too large to serialize. Expected improvement: map() → 50M model loads. mapPartitions(500) → 500 model loads. Speedup: 100,000x reduction in model loading overhead. Target 50 million / 500 partitions = 100K records per partition, highly parallelizable.

Q26 Design ★ Hard Swiggy — Data Engineering Platform Team

Design a PySpark pipeline using mapPartitions to batch-insert 200 million daily order records into a Cassandra cluster. Consider: connection management, error handling, batch size tuning, and idempotency.

 Hint: *Cassandra performs best with batch inserts. Think about partition-level connection pooling and retry logic.*

✅Model Answer (Key Points to Cover):

Design: (1) Partition tuning: repartition to ensure each partition has ~100K records (good Cassandra batch size). (2) mapPartitions function: def cassandra_writer(records): cluster = Cluster(cassandra_hosts); session = cluster.connect('orders'); batch = []; for r in records: batch.append(r); if len(batch) >= 500: session.execute(batch_statement(batch)); batch = []; if batch: session.execute(batch_statement(batch)); cluster.shutdown(). (3) Error handling: Wrap in try/except, log failed batches to a dead-letter RDD using accumulators to track failure count. (4) Idempotency: Use Cassandra's 'IF NOT EXISTS' or set a TTL-based deduplication key so re-runs don't create duplicates. (5) Connection pooling: Use a module-level connection factory with lazy initialization to reuse connections across mapPartitions calls on the same executor.

Q27 Conceptual ★ Medium Accenture — Architecture Round

What is mapPartitionsWithIndex() in PySpark? Give a scenario where the partition index is critical for your processing logic.

 Hint: *Think about when you need to know WHICH partition number you're in.*

✅Model Answer (Key Points to Cover):

mapPartitionsWithIndex(func) is like mapPartitions but passes (partition_index, iterator) to your function, so you know the partition number. Real scenario: A bank processes 28 state-wise data files, one per partition. Partition 0 = Andhra Pradesh data, partition 1 = Telangana, etc. The processing logic differs by state (different tax rules, different holiday calendars). With mapPartitionsWithIndex,

you look up the state-specific config using the partition index: state_config = state_configs[partition_index]. Another scenario: Writing to state-specific S3 paths inside each partition — you need the index to build the correct output path. Without the index, you'd need to embed state information in every record.

Q28 | **Scenario** | **★ Hard** | Ola — Senior Data Engineer Round

Ola has a PySpark job that runs NLP (Natural Language Processing) on 100 million customer feedback records. The NLP library (spaCy with a 500MB English model) is used via mapPartitions. The job runs on a 20-node cluster. The job works locally but throws MemoryError in production. Diagnose and fix this issue.

Hint: *500MB model × multiple executors per node = memory explosion. Think about executor memory config.*

✅Model Answer (Key Points to Cover):

Diagnosis: If each node runs 4 executors, and each loads a 500MB model, that's 4 × 500MB = 2GB just for models per node. Add JVM overhead, Spark executor memory, shuffle buffers — easily exceeds node memory. Fixes: (1) Reduce executors per node: --num-executors-per-node 1 or --executor-cores 4 (fewer, fatter executors). (2) Increase executor memory: --executor-memory 8G to accommodate model. (3) Cache model at executor level (not partition): use a global dict in the worker module — check if model loaded before re-loading. (4) Model broadcasting: Serialize model, broadcast via sc.broadcast(), reconstruct in mapPartitions. (5) Alternative: Use foreachPartition to send data to a model serving microservice (TF Serving/Triton) — keeps heavy model outside Spark entirely. This is a real production architecture decision at ML-heavy companies.

Section 6: Advanced Mixed Scenarios

12 Questions | System Design · Performance · Debug · Architecture

Q29 Scenario ★ Hard Goldman Sachs — Data Engineering Interview

You have a PySpark job that joins trade data (500GB) with reference data (100MB). The job is deployed on a 50-node cluster but takes 4 hours. After checking Spark UI, you see: 45 minutes of GC time, 2 hours of shuffle read time, 1.2 hours of compute. What are the top 3 optimization steps?

Hint: *GC = too many objects / memory pressure. Shuffle time = large shuffle write. Analyze each bottleneck independently.*

✅Model Answer (Key Points to Cover):

(1) Fix Shuffle (2hrs): Convert the join to a Broadcast Join. Reference data (100MB) easily fits in memory: ref_bc = sc.broadcast(dict(ref_rdd.collect())). Eliminates ALL shuffle for the join step — from 2hrs to near-zero. (2) Fix GC (45min): Use Kryo serialization instead of Java default: conf.set('spark.serializer', 'org.apache.spark.serializer.KryoSerializer'). Reduce partition count (too many small objects). Use primitive arrays instead of complex objects in RDD. Consider off-heap storage with Tungsten. (3) Fix Compute (1.2hrs): Profile the UDF or transformation logic. Use DataFrames instead of RDD — Catalyst optimizer and Tungsten engine can reduce compute by 5-10x for common operations. Expected improvement: 4 hours → under 30 minutes.

Q30 Debug ★ Hard Deloitte — Spark Production Incident

A PySpark job at an NBFC (Non-Banking Finance Company) ran successfully in UAT (50K records) but crashes in production (50 million records) with: 'org.apache.spark.SparkException: Job aborted due to stage failure: Total size of serialized results of 1048576 tasks (1024.0 GB) is bigger than spark.driver.maxResultSize (1.0 GB)'. What is the root cause and how do you fix it?

Hint: *The driver is collecting too much data. Which action causes this?*

✅Model Answer (Key Points to Cover):

Root cause: A collect() or similar action (like countByKey(), collectAsMap()) is being called on a 50M record RDD. All results are serialized and sent to the driver, exceeding its 1GB limit. Fix options: (1) Increase limit: --conf spark.driver.maxResultSize=4g (temporary workaround, not scalable). (2) Avoid collect() on large RDDs: Instead of rdd.collect() → process on driver, use rdd.foreachPartition() to push data directly to the destination from executors. (3) If you need count: use rdd.count() (returns one integer) instead of countByKey() (returns all keys to driver). (4) If you need to save: use saveAsTextFile(), not collect(). Root cause in code: Find every .collect(), .take(), .countByKey(), .collectAsMap() call and evaluate if it's truly needed on the driver.

Q31 Design ★ Hard Zepto — Data Platform Architecture

Zepto does 10-minute delivery. Their data pipeline must process orders in real-time and update: (a) live inventory (Cassandra), (b) delivery partner location matching (Redis), (c) daily analytics (HDFS). Design a PySpark Structured Streaming pipeline using foreach/foreachPartition/mapPartitions that handles 100K orders/minute with < 5 second end-to-end latency.

 Hint: *Think about micro-batch size, connection pooling per partition, and parallel writes to 3 sinks.*

✅Model Answer (Key Points to Cover):

Pipeline design: Trigger interval: 5-second micro-batches (processingTime='5 seconds'). foreachBatch function: def process_batch(df, epoch_id): rdd = df.rdd.repartition(50) → rdd.cache(). Branch 1 — Inventory (Cassandra): rdd.foreachPartition(cassandra_inventory_update) — one Cassandra session per partition, batch statements of 100 records. Branch 2 — Partner matching (Redis): rdd.foreachPartition(redis_location_publish) — one Redis connection per partition using pipeline mode (100 commands pipelined). Branch 3 — Analytics (HDFS): rdd.coalesce(10).saveAsTextFile(f'/data/{epoch_id}'). rdd.unpersist() after all 3 writes. Latency budget: 5s trigger + ~3s processing = 8s worst case — tune micro-batch to 2s and executor memory to meet 5s SLA. This is a real Zepto/Blinkit-scale architecture discussion.

Q32 | **Conceptual** | **★ Medium** | Barclays India — Data Engineering

Explain the concept of narrow vs wide transformations in PySpark. Classify: map, filter, flatMap, groupByKey, reduceByKey, join, coalesce, repartition, union, mapPartitions.

 Hint: *Narrow = one-to-one partition dependency, no shuffle. Wide = many-to-one, triggers shuffle.*

✅Model Answer (Key Points to Cover):

Narrow transformations (NO shuffle, fast): map(), filter(), flatMap(), mapPartitions(), union() — each output partition depends on exactly ONE input partition. Wide transformations (SHUFFLE, expensive): groupByKey(), reduceByKey(), join(), sortByKey(), distinct(), repartition() — output partition depends on MULTIPLE input partitions, requires data movement across nodes. Special cases: coalesce(decrease) = narrow (merges locally), coalesce(increase) = treated as repartition (wide). union() is narrow — it just concatenates partition lists. Interview insight: Wide transformations create stage boundaries in the DAG. Each shuffle = one stage. Minimizing wide transformations = minimizing stages = faster jobs.

Q33 | **Scenario** | **★ Hard** | Juspay — Payment Gateway Interview

Juspay processes international payments and must apply currency conversion to 500 million transaction records. The exchange rates change every minute and are stored in a REST API. Using mapPartitions, design a solution that fetches fresh rates for each micro-batch without fetching per-record, while ensuring currency rates are no more than 1 minute stale.

 Hint: *One API call per partition per micro-batch, not per record. Think about rate staleness.*

✅Model Answer (Key Points to Cover):

Solution: Use mapPartitions with API call at partition level. def apply_fx_rates(records): import requests; response = requests.get('https://forex-api/rates', timeout=5); rates = response.json(); for

txn_id, amount, currency in records: converted = amount * rates.get(currency, 1.0); yield (txn_id, converted, currency). With Structured Streaming (1-minute trigger): each partition fetches rates ONCE per batch. With 200 partitions and 1-minute batches: 200 API calls/minute instead of 500M/minute. Staleness: Since trigger = 1 minute and rates change every minute, worst case staleness = 1 minute. To reduce: shorten trigger to 30 seconds. Fallback: Cache last known rates in a broadcast variable, update the broadcast at batch start on driver, fall back to broadcast if API fails.

Q34 | **Design** | **★ Hard** | CRED — Data Engineering Architecture

You are building CRED's credit score refresh pipeline. Every night, 40 million user profiles must be enriched with: (1) latest transactions (join), (2) bureau score (mapPartitions + external API), (3) spending category analysis (groupByKey + reduceByKey), (4) result written to 3 systems. Design the complete RDD pipeline with partition strategy, caching points, and execution order.

Hint: *Identify where to cache, where to repartition, and which operations are most expensive.*

✅Model Answer (Key Points to Cover):

Pipeline design: Step 1: Load user_rdd (40M users) and txn_rdd (200M transactions). repartition(400) both on user_id → ensures collocated data for join. Step 2: Join user_rdd.join(txn_rdd) → enriched_rdd. cache() enriched_rdd — used by steps 3, 4, 5. Step 3: Bureau score via mapPartitions: one API call per partition (400 API calls total). Step 4: Spending analysis: enriched_rdd.map(extract_category).reduceByKey(aggregate) → category_rdd. Step 5: Write to 3 sinks: enriched_rdd.foreachPartition(write_to_cassandra), category_rdd.coalesce(1).saveAsTextFile(s3_summary), enriched_rdd.foreachPartition(write_to_elasticsearch). Step 6: enriched_rdd.unpersist(). Execution order matters: cache before forking to 3 sinks. Without cache, full pipeline replays 3 times.

Q35 | **Debug** | **★ Medium** | Mu Sigma — Data Engineering Round

A junior developer's PySpark job uses union() to combine 365 daily RDDs (one per day of the year) in a for loop. The job runs fine for 30 days but crashes with a StackOverflowError for 365 days. What is the problem and how do you fix it?

Hint: *Think about what repeated union() calls in a loop create in terms of the DAG.*

✅Model Answer (Key Points to Cover):

Root cause: Each union() creates a new RDD with a reference to both parent RDDs. After 365 unions, the logical DAG is 365 levels deep — an extremely deep nested tree. When Spark tries to serialize/analyze this DAG for optimization or checkpointing, it recursively traverses it and hits Java's stack depth limit (StackOverflow). Fix 1 (Best): Use SparkContext.union() which accepts a list: all_days_rdd = sc.union(daily_rdds_list). This creates a flat union with ONE level of depth. Fix 2: If loading from files, use sc.textFile('/path/year=2024/*') to load all 365 days in one call. Fix 3: Periodically checkpoint the RDD every N days to truncate the lineage: rdd.checkpoint() after every 30 days in the loop. The sc.union(list) approach is the standard production pattern.

Q36 | **Scenario** | **★ Hard** | Navi Technologies — Data Platform Interview

Navi's loan approval pipeline joins applicant data (5M records) with 8 different reference tables (credit bureau, income tax records, GST data, property records, employer records, bank statements, CIBIL, court records). Each join adds columns. The final dataset is used for ML scoring. The current pipeline takes 9 hours. Redesign it.

Hint: *8 separate joins = 8 shuffles. Think: which tables are small enough to broadcast? Sequence matters.*

✅Model Answer (Key Points to Cover):

Optimization strategy: Step 1: Classify reference tables by size — CIBIL, credit bureau, income tax might be 100MB-1GB; property, GST, court records might be smaller. Step 2: Broadcast all tables under 500MB: ref_bc_1 = sc.broadcast(dict(table1.collect())) etc. This eliminates their shuffle entirely. Step 3: For remaining large tables, sort-merge join: ensure both sides are co-partitioned on applicant_id — use repartition(applicant_id) once, then all joins use the same partitioning (only 1 shuffle per large table instead of per join). Step 4: Use mapPartitions to apply all broadcast lookups in ONE pass instead of 8 separate joins: for each record, look up all 8 broadcast tables together. Result: from 8 shuffles to 1-2 shuffles + broadcast lookups. Expected time: 9 hours → 45-90 minutes.

Q37 | **Conceptual** | **★ Medium** | Razorpay / HDFC — System Design

Explain DAG (Directed Acyclic Graph) in PySpark. How does Spark use it? How does lazy evaluation relate to it? What happens when you call an action?

Hint: *Think: transformation builds the DAG, action triggers execution. Driver submits DAG to scheduler.*

✅Model Answer (Key Points to Cover):

DAG in Spark: When you write transformations (map, filter, join etc.), Spark does NOT execute them — it builds a DAG (logical execution plan) in memory on the driver. Each transformation adds a node to the DAG. Lazy evaluation: Spark waits until an ACTION (collect, count, save) is called before executing anything. Benefits: Spark can OPTIMIZE the entire pipeline before running — it can combine adjacent map() calls, push filters down, choose broadcast joins automatically. When action is called: (1) DAG is submitted to DAGScheduler. (2) DAGScheduler splits into Stages (each wide transformation = stage boundary). (3) Stages split into Tasks (one per partition). (4) TaskScheduler assigns tasks to executors. (5) Results return to driver. This is why Spark can be 100x faster than Hadoop — entire job is optimized as one unit.

Q38 | **Design** | **★ Hard** | Groww — Data Engineering Lead Interview

Groww processes stock market data from NSE and BSE feeds simultaneously. Design a pipeline that: reads from both feeds (two RDDs), identifies stocks traded on both exchanges (intersection), stocks only on NSE (complement), computes per-stock total volume using mapPartitions for heavy computation, and writes the 3 outputs to separate HDFS paths with appropriate partition strategies.

Hint: *Use set operations (subtractByKey, join for intersection), then mapPartitions for computation.*

✅Model Answer (Key Points to Cover):

Pipeline: Load: nse_rdd = (symbol, price, volume) and bse_rdd = (symbol, price, volume). cache() both — used multiple times. Intersection (both exchanges): nse_rdd.join(bse_rdd) → traded_on_both. NSE only: nse_rdd.subtractByKey(bse_rdd) → nse_only. Volume computation with mapPartitions: def compute_volume_stats(partition): for symbol, data in partition: yield (symbol, heavy_computation(data)). Apply to traded_on_both.mapPartitions(compute_volume_stats). Write strategy: (a) traded_on_both: repartition(10).saveAsTextFile('/hdfs/both') — moderate file count. (b) nse_only: coalesce(5).saveAsTextFile('/hdfs/nse_only') — smaller dataset. (c) volume_stats: coalesce(1).saveAsTextFile('/hdfs/volume_stats') — summary, one file. nse_rdd.unpersist(); bse_rdd.unpersist() after all writes.

Q39 | **Scenario** | **★ Hard** | Dream11 — Data Engineering Interview

Dream11 runs contests with 10 million participants simultaneously. During match scoring, player points must be calculated and leaderboards updated every 10 seconds. The computation uses mapPartitions (heavy scoring logic). The job keeps failing at the 'update leaderboard' step with executor OOM errors. Diagnose and resolve.

Hint: *OOM in mapPartitions = partition too large OR holding too many objects in memory. Think: partition sizing and iterator laziness.*

✅Model Answer (Key Points to Cover):

Diagnosis: Two likely causes — (1) Partition size too large: 10M records / current partitions = oversized. mapPartitions loads entire partition processing in memory if not using generators (yield). (2) Materializing entire partition: If the function collects all records into a list instead of using lazy yield, the full partition sits in memory. Fix 1 — Partition sizing: Increase partitions: repartition(500) → 20K records per partition, manageable. Fix 2 — Use generators: Ensure mapPartitions function uses yield (lazy) not return list (eager). WRONG: def score(records): return [compute(r) for r in records]. RIGHT: def score(records): for r in records: yield compute(r). Fix 3 — Executor memory tuning: --executor-memory 8G --executor-cores 2 (more memory per core). Fix 4 — GC tuning: Use G1GC: --conf spark.executor.extraJavaOptions=-XX:+UseG1GC. With generators + right partition size, OOM should resolve.

Q40 | **Design** | **★ Hard** | Google/Microsoft — Distinguished Engineer Round

You are given an RDD of 10 billion web clickstream events from across India. Each event is (user_id, page_url, timestamp, device_type). Design a complete analytics pipeline using appropriate RDD operations to produce: (a) top 1000 pages by traffic, (b) unique users per device type, (c) user journey sequences (ordered clicks per user). Justify every design decision with performance implications.

Hint: *This is an open-ended design question. There is no single right answer — justify your choices.*

✅Model Answer (Key Points to Cover):

(a) Top 1000 pages: clicks_per_page = rdd.map(lambda x: (x[1], 1)).reduceByKey(add).sortBy(lambda x: -x[1]).take(1000). Avoid groupByKey (high memory). sortBy + take is efficient — doesn't materialize full sorted RDD. (b) Unique users per device: rdd.map(lambda x: (x[3], x[0])).distinct().map(lambda x: (x[0], 1)).reduceByKey(add).collect(). distinct() before count

avoids counting same user twice per device. (c) User journeys: rdd.map(lambda x: (x[0], [(x[2], x[1])])).reduceByKey(lambda a,b: a+b).mapValues(lambda v: [url for ts,url in sorted(v)]).saveAsTextFile(path). Challenge: reduceByKey collects all clicks per user — for power users with millions of clicks, this causes skew. Fix: Use repartitionAndSortWithinPartitions to sort by (user_id, timestamp) without full collect, then use mapPartitions to group by user. This avoids materializing full user history.